Sprint 2 (20 points)

Part I. README and CI/CD Pipeline (5 points)

1. README (2 points)
Your project must include a README.md at the root of the repo that clearly describes the software and how to use it.
Specifically, the README should outline key features, provide clear instructions for installation, setup, and how to run the system, along with simple usage examples to help users get started. Including screenshots or snapshots is recommended to illustrate the system in use.
The README may also mention known issues or limitations, and provide links to additional resources (such as API documentation or a project wiki) if applicable.

2. CI/CD Pipeline (3 points)
Your project must have a working CI/CD pipeline that automatically builds and possibly deploys your software.
Required steps in the pipeline:
compile the source code
run tests
package the application into a runnable artifact
Optional steps to enhance your pipeline:
running linters
generating test reports
generating documentation
building a Docker image
pushing the image to a container registry (e.g., Docker Hub)
deploying to a Kubernetes cluster
running the application in docker containers
The CI/CD pipeline should be triggered by commits to the main branch, and it should provide feedback or logs on the build status (e.g., success or failure) through GitHub Actions or other CI/CD tools.
Heads up: our sustech-cs304 github organization account has hit some usage limits, and due to regional payment restrictions, a few GitHub features (like storage operations or GitHub Actions on private repos) might not work reliably for everyone.

Yet, you still have a couple of easy ways to complete this requirement:
Use another CI/CD tool (e.g., Jenkins) and run your pipeline locally.
If you want to stick with GitHub Actions for CI/CD, you can:

Make your repository public, or
Fork the repo to your personal GitHub account and continue there. GitHub Free

accounts come with a monthly quota of CI/CD minutes, which should be enough for this sprint. If you use this option, you need to explicitly provide the forked repo's URL in the team report.

Part II. Team Report (4 points)
You are required to write a team report, which includes the following information.

1. Metrics (2 points)
We could use metrics to describe the complexity of your project. Please compute and report the following metrics (you could use whatever tools or libraries for computing the metrics):
Lines of Code
Number of source files
Cyclomatic complexity
Number of dependencies

2. CI/CD Pipeline Description (2 points)
Describe the CI/CD pipeline of your project, including the following information:
Steps in the pipeline
Tools/technologies/frameworks used for implementing each step of the pipeline (e.g., JUnit and JaCoCo for testing and test coverage report generation)
In the report, you should also provide:
Access to the pipeline configuration (e.g., workflow files, scripts) via URL links or snapshots
A proof that the pipeline executes successfully (you may provide screenshots or logs).

Part III. AI Usages (1 point)

Complete this survey to report your AI usages for documentation, tests, computing metrics, and CI/CD. One submission per team is enough.
Also, same as the previous sprint, you should use git properly for version control and collaborations. Failures to do so will incur score penalties.
Part IV. Sprint Review (10 points)
In this final presentation, your team should demonstrate the entire running system, focusing on features and usability.
Imagine that this demo will be presented to end users, clients, and potential investors. Your team should demonstrate all notable features of the system, with the purpose of attracting users/investors to use/invest the software system.

During the demo
Start you application by the executable or from the container.
Functional features: Your project must include at least five distinct and meaningful features, and you should clearly introduce them one by one (e.g., “Feature 1 is…”, “Feature 2 is…”, etc.). You may also record a video beforehand, if some features

depend on external factors (e.g., user input, network conditions, non-deterministic LLM outputs, etc.) that are hard to control during the demo.
Non-functional features: if applicable, briefly demonstrate your efforts on improving the non-functional features of your system, such as performance, security, or user experience.
Basic requirements (up to 50% points will be deducted for failing these requirements):
Each team will give a presentation in the lab session of week 14 (+0.3 bonus) or 15.
Every team member needs to show up during the presentation.

Submissions
We will check your team repo for the code, scripts, other artifacts, and the commit history of your project.
You should also submit the team report as final-report-teamID.md to the team repo.